

Reviewer Pack — Schur Coefficient Exponential Gap in Symmetric Squares of Pe...

Entry 9b651f40d7fc · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-20 06:41:45 UTC

Schur Coefficient Exponential Gap in Symmetric Squares of Permutation Representations for Random CNFs

Entry ID: 9b651f40d7fc **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-20 06:41:45 UTC

1. Conjecture

Field A (mathematical branch): Representation Theory of Symmetric Groups **Field B** (complexity object): Boolean Circuit Complexity of Random CNFs

Statement:

Let f be a homogeneous polynomial of degree n^2 in n variables with coefficients sampled uniformly from $\{-1, 1\}$. Let $S^2(f)$ denote its symmetric square. The Schur coefficient of the permutation representation $(n-1, 1)$ in $S^2(f)$ is $\Omega(2^{\{n/2\}})$ for random f , while for the determinant polynomial $\det_n$, this coefficient is $O(n^3)$.

Rationale (proposer's reasoning):

Schur coefficients capture the multiplicity of irreducible representations in symmetric powers, revealing structural asymmetry between permutation-invariant polynomials and determinants. This connects representation-theoretic invariants to circuit complexity via the orbit method, avoiding natural proofs barriers through explicit algebraic structure.

Taxonomy category: META_COMPLEXITY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 5d58422078dd9b87

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        n = len(A)
        for i in range(n):
            max_row = i
            for j in range(i+1, n):
                if abs(A[j][i]) > abs(A[max_row][i]):
                    max_row = j
            A[i], A[max_row] = A[max_row], A[i]
            pivot = A[i][i]
            for j in range(n):
                A[i][j] /= pivot
            for j in range(n):
                if j != i:
                    factor = A[j][i]
                    for k in range(n):
                        A[j][k] -= factor * A[i][k]
        return A

    def matrix_multiply(A, B):
        n = len(A)
        C = [[0]*n for _ in range(n)]
        for i in range(n):
            for j in range(n):
                for k in range(n):
                    C[i][j] += A[i][k] * B[k][j]
        return C

    def schur_coefficient(A, n):
        identity = [[1 if i == j else 0 for j in range(n)] for i in range(n)]
        symmetric_square = matrix_multiply(matrix_multiply(A, A), identity)
        det_A = 1
        for i in range(n):
```

```

        det_A *= symmetric_square[i][i]
    return abs(det_A) ** (1/n)

n = random.randint(5, 40)
A = [[random.choice([-1, 1]) for _ in range(n)] for _ in range(n)]
schur_coeff = schur_coefficient(A, n)

det_n = math.factorial(n) / (math.factorial(n-2) * 2)

return {
    "metric_name": "Schur Coefficient",
    "metric_value": schur_coeff,
    "instances_tested": 1,
    "conjecture_holds": schur_coeff >= 2**(n/2),
    "counterexample": "" if schur_coeff >= 2**(n/2) else f"det_n={det_n}, schur_coeff={schur_coe
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(100, 999) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fra
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fra
    else:
        for r in results:
            if not r["conjecture_holds"]:
                print(f"RESULT: FALSIFIED counterexample=\"{r['counterexample']}\" first_failing_s
                break

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

erexample': 'det_n=153.0, schur_coeff=0.0'}
TRIAL: {'metric_name': 'Schur Coefficient', 'metric_value': 2.6364449413756286, 'instances_tested': 1}
TRIAL: {'metric_name': 'Schur Coefficient', 'metric_value': 1.4723567001803468, 'instances_tested': 1}
TRIAL: {'metric_name': 'Schur Coefficient', 'metric_value': 3.2208542681806756, 'instances_tested': 1}
TRIAL: {'metric_name': 'Schur Coefficient', 'metric_value': 3.889872159407338, 'instances_tested': 1}
TRIAL: {'metric_name': 'Schur Coefficient', 'metric_value': 1.8949164306617055, 'instances_tested': 1}
TRIAL: {'metric_name': 'Schur Coefficient', 'metric_value': 0.0, 'instances_tested': 1, 'conjecture_holds': False}
TRIAL: {'metric_name': 'Schur Coefficient', 'metric_value': 0.0, 'instances_tested': 1, 'conjecture_holds': False}
TRIAL: {'metric_name': 'Schur Coefficient', 'metric_value': 0.0, 'instances_tested': 1, 'conjecture_holds': False}

```

TRIAL: {'metric_name': 'Schur Coefficient', 'metric_value': 2.784154792775127, 'instances_tested': 1
 TRIAL: {'metric_name': 'Schur Coefficient', 'metric_value': 3.0014468371076175, 'instances_tested': 1
 RESULT: FALSIFIED counterexample="det_n=528.0, schur_coeff=2.9279614430458305" first_failing_seed=9

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

{ "critic_verdict": "CHALLENGE", "reasoning": "n is too small: The test uses det_n=300 (implying n=10) but claims $\Omega(2^{\{n/2\}}) = \Omega(32)$." }

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge_falsified | original: The test found counterexamples where schur_coeff values (≤ 4) are orders of magnitude smaller than the conjectured $\Omega(2^{\{n/2\}})=\Omega(32)$ for n=10. | next: Analyze the scaling behavior for larger n to determine if the exponential gap emerges or if the conjecture requires revision.

11. Audit log (LLM calls)

Total LLM calls: 12

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	51602
2	propose	ollama_remote	qwen3:8b	0	0	138563
3	propose	ollama_remote	qwen3:8b	0	0	41586
4	preregistration	ollama_remote	qwen3:8b	0	0	31985
5	novelty	ollama_remote	qwen3:8b	0	0	27597
6	novelty	ollama_remote	qwen3:8b	0	0	23276
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15876
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6577
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11103
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8990
11	critic	ollama_remote	qwen3:8b	0	0	38548
12	judge	ollama_remote	qwen3:8b	0	0	23991

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 419693 ms total latency. Provider mix: {'ollama_remote': 12}

(full prompt+response transcripts available in `research/audit/9b651f40d7fc.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/9b651f40d7fc.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/9b651f40d7fc.tar.gz (if generated)